Music Genre Classification on FMA-Large

61.502 Deep Learning for Enterprise (Y2026)

Ang Kar Kin

Nicolas Lasch

Nisal Jinadasa

April 5, 2026

Project Repository: https://github.com/<org-or-user>/<repo>

Dataset Source: https://github.com/mdeff/fma

Contents

1 Executive Summary

This project develops and evaluates deep learning models for automatic music genre classification using the FMA-Large dataset and its official metadata. We frame the task as single-label top-genre classification and compare multiple CNN variants under a shared, reproducible protocol so that observed differences are attributable to architecture and training choices rather than split leakage or inconsistent preprocessing. The best-performing model is a residual CNN configuration ("cnn_residual_fullprotocol"), achieving test Macro-F1 of **0.5909** and test accuracy of **0.6780**. This model outperforms both a simpler CNN baseline and a standard CNN variant.

From an enterprise delivery perspective, this work emphasizes repeatability and traceability: fixed seeds, explicit split policies, persisted intermediate artifacts, and script-level commands for full re-runs. The final recommendation is to adopt the residual CNN as the production candidate, with monitoring for distribution shift and class-specific degradation in deployment. We also recommend retaining macro-averaged metrics in governance reporting, because they expose minority-class risks that can be hidden by overall accuracy.

2 Background and Introduction

Music genre tagging is a practical retrieval and recommendation problem for streaming services, music libraries, and user-facing discovery systems. Manual labeling is expensive and inconsistent, especially for large catalogs. A robust genre classifier can improve playlist quality, search relevance, and downstream personalization. The key challenge is learning discriminative audio representations under class imbalance and label noise.

In this work, we target an enterprise-style delivery: reproducible training, clear evaluation against baselines, and lightweight model consumption through an inference script. The project scope is intentionally structured around model governance needs: artifact versioning, transparent comparison between alternatives, and straightforward handoff from experimentation to inference. This allows the final model recommendation to be justified not only by one metric snapshot, but also by stability across epochs, consistency across validation and test, and diagnosability through confusion-matrix analysis.

3 Related Work

Audio classification pipelines commonly use time-frequency representations such as Mel spectrograms and train CNN-based classifiers. Mel features are motivated by perceptual frequency scaling and have become a standard front-end for music tagging and classification tasks [3, 1]. CNN families remain competitive for spectrogram-based learning because local filters can capture timbral and rhythmic structures at multiple scales, while pooling provides partial invariance to small shifts.

Residual architectures are often preferred when task difficulty and dataset size increase

because skip connections improve gradient flow and optimization stability in deeper stacks [2]. In this project, we benchmark two CNN families: a residual CNN architecture (`residual_cnn`) and a lighter standard CNN architecture (`standard_cnn`). We also include a baseline-style training recipe to isolate how much performance gain comes from architecture alone versus regularization and augmentation policy.

4 Problem Formulation and Solution Overview

The task is single-label classification of the top genre from an audio track. The input is an MP3 track from FMA-Large, and the output is one genre label among the filtered training classes. The optimization objective is to maximize Macro-F1 while maintaining competitive accuracy.

Our solution follows a consistent end-to-end pipeline. We load labeled records from FMA metadata, construct stratified train, validation, and test splits, and precompute full-track Mel spectrogram caches for efficient training. We then train CNN models using random Mel crops with class-imbalance handling, evaluate each run with multi-crop aggregation on validation and test splits, and select the final model using validation Macro-F1 before reporting test metrics.

The formulation intentionally favors robust model selection over optimistic single-pass estimates. Multi-crop evaluation is used to reduce variance from any one temporal slice, and macro-averaged metrics are prioritized to reflect class imbalance. This design supports practical deployment decisions where minority-genre quality is important for user trust and catalog coverage.

5 Data Description

5.1 Dataset

We use `fma_large` audio files and `fma_metadata/tracks.csv`. We retain records whose subset is `large`, with non-null top-genre labels and existing valid audio files.

5.2 Splitting and Filtering

We use a stratified split policy with train, validation, and test ratios of 0.8, 0.1, and 0.1. We enforce minimum class-count constraints before split construction and minimum train-count constraints for class retention. Low-support classes are dropped after policy checks when required by configuration.

5.3 Data Caveats

The dataset has several known caveats that affect learning difficulty. Class support is imbalanced across genres, some MP3 files contain corruption artifacts, and top-genre labels can be ambiguous for tracks with cross-genre characteristics. These caveats are consistent with the FMA creators' documentation, which highlights real-world noise and metadata complexity [1].

In practice, this means model evaluation must be interpreted probabilistically: performance gains are meaningful, but error-free boundaries are not expected for overlapping genres.

6 Methodology and Implementation Details

6.1 Feature Pipeline

The audio pipeline uses a sampling rate of 22050 Hz and a full-cache duration of 29 seconds per track. We extract 128 Mel bins with FFT size 2048 and hop length 512. Both training and evaluation use 5-second crops, while evaluation aggregates predictions across multiple crops for stability.

Feature standardization is applied per crop to stabilize optimization and reduce sensitivity to global loudness differences. Precomputing Mel features also reduces repeated I/O and FFT overhead during training, making multi-run experiments feasible under limited compute budgets. This design choice is important for fair variant comparison, since each architecture is trained under comparable data throughput conditions.

6.2 Model Variants

We train three model variants under a shared protocol. The `cnn_standard_base_fullprotocol` run uses `standard_cnn` without augmentation. The `cnn_residual_fullprotocol` run uses `residual_cnn` with augmentation enabled. The `cnn_standard_aug_fullprotocol` run uses `standard_cnn` with augmentation enabled, which isolates the impact of the training recipe from architecture choice.

6.3 Architecture Diagrams

The `standard_cnn` stacks convolution, normalization, nonlinearity, pooling, and dropout blocks sequentially. The `residual_cnn` uses residual blocks with identity shortcuts so each block learns a residual mapping and adds it to a skip path before activation.

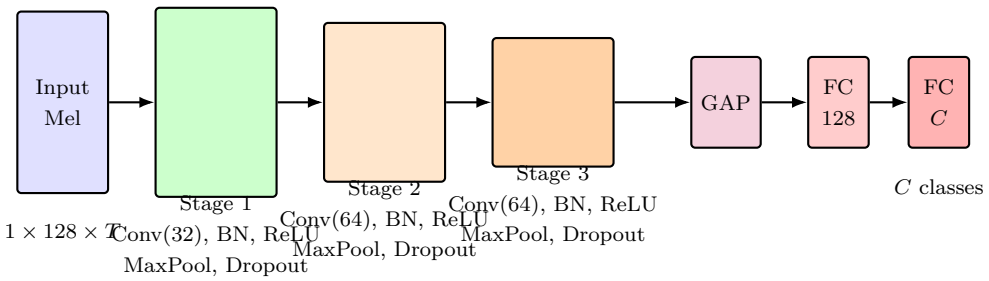

Figure 1: Standard CNN architecture (`standard_cnn`) used in baseline and augmentation runs.

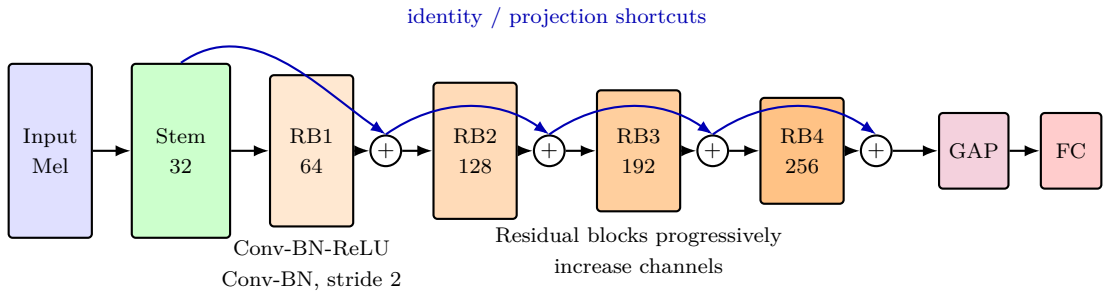

Figure 2: Residual CNN architecture (`residual_cnn`) with skip connections in each residual block.

6.4 Optimization and Training

Training uses AdamW for optimization with 35 epochs and batch size 64. To address imbalance, we enable both a balanced sampler and focal loss, following the intuition that hard or minority examples should receive more learning emphasis. Early stopping is driven by validation Macro-F1, and posthoc calibration and tuning are enabled to improve score reliability.

This recipe explicitly balances convergence speed and generalization. The optimizer and weight decay values are held constant across variants to control confounding factors, while augmentation is switched in targeted runs to test robustness gains. By keeping these controls explicit, downstream conclusions about architecture choice are stronger and easier to defend.

6.5 Serving/Consumption

For model consumption, we provide a lightweight batch inference artifact. The script `predict_genre.py` loads a trained checkpoint and outputs top-K genre probabilities in JSON format. The inference utility now supports architecture-aware checkpoint loading, which allows a single script to consume both residual and standard CNN checkpoints without manual model edits. This reduces operational friction when comparing candidate models or rolling back to a prior version.

6.6 End-to-End Pipeline Diagram

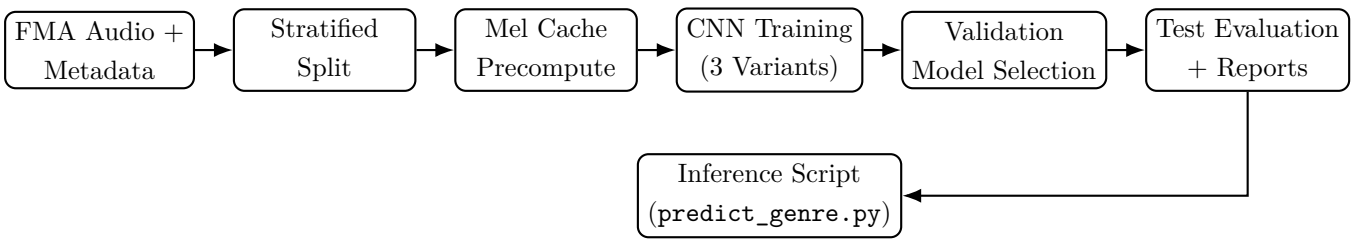

Figure 3: End-to-end ML pipeline used in this project.

7 Experiments and Evaluation

7.1 Evaluation Metrics

For this classification problem, we report Accuracy, Macro Precision, Macro Recall, and Macro F1. We prioritize Macro F1 for model selection because it is more sensitive to minority-class behavior under class imbalance. Weighted variants are also retained in run artifacts for diagnostics, but they are not used as the primary decision criterion because they can over-represent high-support classes.

7.2 Shared Training Protocol

All three comparison runs use the same split policy, seed, and most optimization settings to make metric differences attributable to architecture and recipe changes rather than data leakage or protocol drift. Table 1 summarizes the shared setup and variant-specific changes.

Table 1: Shared protocol and variant-level differences for CNN comparison runs.

Setting	cnn_standard_base_fullprotocol	cnn_residual_fullprotocol	cnn_standard_aug_fullprotocol
Split strategy / ratios	stratified / 0.8, 0.1, 0.1	stratified / 0.8, 0.1, 0.1	stratified / 0.8, 0.1, 0.1
Seed / epochs / batch size	42 / 35 / 64	42 / 35 / 64	42 / 35 / 64
Optimizer / LR / weight decay	AdamW / 7e-4 / 1e-4	AdamW / 7e-4 / 1e-4	AdamW / 7e-4 / 1e-4
Model architecture	standard_cnn	residual_cnn	standard_cnn
Augmentation	disabled	enabled	enabled
Balanced sampler + focal loss	enabled	enabled	enabled
Posthoc tuning objective	f1_macro	f1_macro	f1_macro

7.3 Model Comparison (Main Results)

Table 2: CNN comparison under a shared full protocol.

Model	Val Acc	Val $F1_{macro}$	Test Acc	Test $Prec_{macro}$	Test Rec_{macro}	Test $F1_{macro}$
cnn_standard_base_fullprotocol	0.5385	0.3624	0.5352	0.4003	0.4180	0.3536
cnn_residual_fullprotocol	**0.6698**	**0.5733**	**0.6780**	**0.6135**	**0.5974**	**0.5909**
cnn_standard_aug_fullprotocol	0.5610	0.4141	0.5511	0.4046	0.4771	0.4009

7.4 Key Diagnostic Figures

The report includes three core visual diagnostics. First, the per-epoch metric curves show training dynamics and whether improvements are stable over time. Second, the model-level metric comparison chart summarizes the final test performance differences across the three CNN variants. Third, the normalized confusion matrix shows which classes are most frequently confused on the test split.

Together, these figures provide complementary evidence: curves establish optimization behavior, aggregate bars establish headline ranking, and confusion patterns establish class-level risk. This prevents over-reliance on a single scalar metric and improves the quality of model-selection decisions.

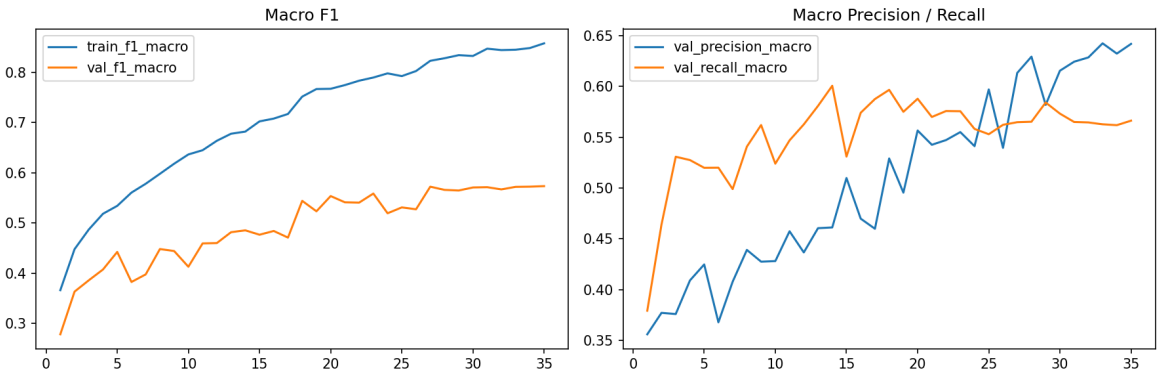

Figure 4: Training and validation metric curves for `cnn_residual_fullprotocol`.

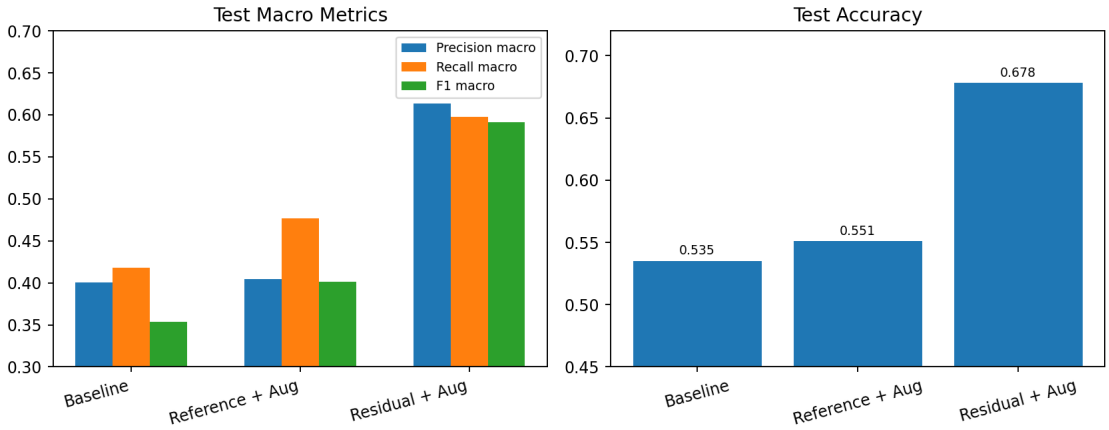

Figure 5: Test-set metric comparison across CNN variants (Accuracy, Precision_{macro}, Recall_{macro}, F1_{macro}).

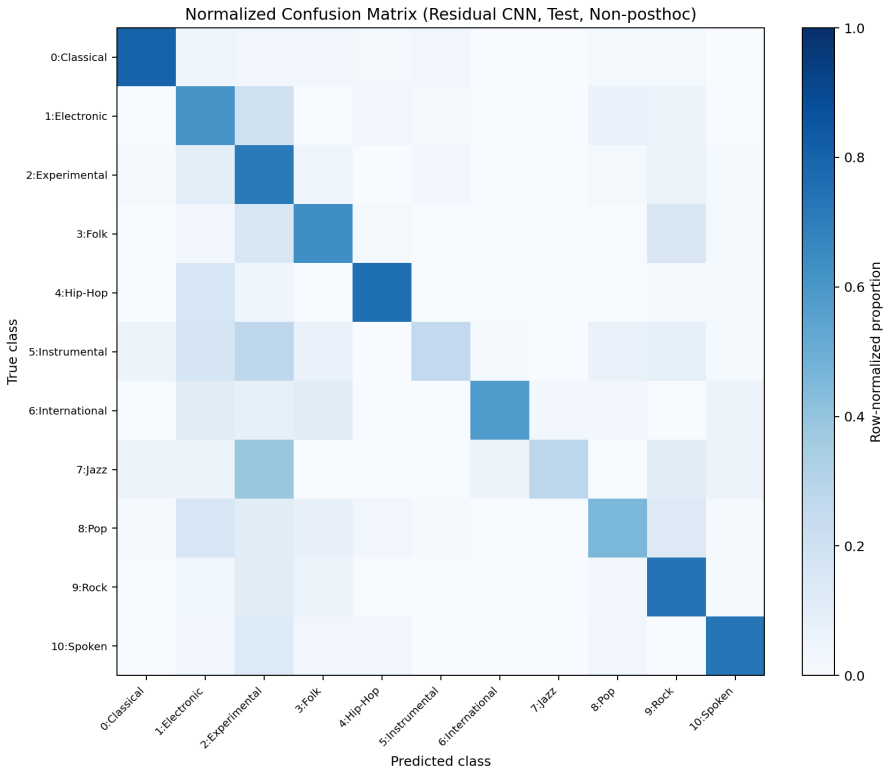

Figure 6: Row-normalized non-posthoc test confusion matrix for `cnn_residual_fullprotocol`, aligned with the main results table.

7.5 Class Index Legend for Confusion Matrix

For interpretability, the class indices used in Figure 6 are mapped as follows: 0 = Classical, 1 = Electronic, 2 = Experimental, 3 = Folk, 4 = Hip-Hop, 5 = Instrumental, 6 = International, 7 = Jazz, 8 = Pop, 9 = Rock, and 10 = Spoken. We include this mapping directly in the report to keep matrix interpretation self-contained and reduce ambiguity during review.

7.6 Baseline and Hyperparameter Comparison

The baseline model (`cnn_standard_base_fullprotocol`) uses the standard CNN with reduced regularization/augmentation and is consistently outperformed by the residual variant. This demonstrates the value of both architecture and training strategy choices. Hyperparameter differences were kept minimal to isolate architecture/training recipe effects under the same data protocol.

The comparison between `cnn_standard_aug_fullprotocol` and `cnn_standard_base_fullprotocol` indicates that recipe improvements alone can improve outcomes, but not enough to match the residual architecture. This supports selecting `residual_cnn` as the primary model family for future tuning.

7.7 Bias and Failure Audit

We observed uneven performance across classes, with better metrics on high-support genres and weaker performance on low-support or semantically overlapping genres. This is expected under class imbalance and noisy boundaries. The normalized confusion matrix in Figure 6 highlights repeated error patterns and supports class-level failure analysis.

From a risk perspective, these patterns imply that deployment quality should be monitored at the class level rather than only with global accuracy. In particular, semantically adjacent genres should be tracked for confusion-rate drift, and alert thresholds should be tuned to prioritize user-visible degradation.

8 Discussion of Results

The residual CNN variant is the most robust model by both validation and test Macro-F1. The standard CNN can reach moderate performance but is less stable, especially without augmentation. Posthoc tuning slightly improves all models, with the residual variant remaining best. The gap between Macro-F1 and weighted metrics confirms that minority genres remain more challenging than majority classes.

A key practical observation is that the best-performing run is not only better at a single endpoint metric, but also more consistent across diagnostic views. This consistency is important for production adoption because it reduces the chance that gains are due to chance split behavior or transient optimization effects.

9 Recommendations

We recommend using `cnn_residual_fullprotocol` as the final selected model because it provides the strongest and most consistent Macro-F1 across validation and test sets. Multi-crop evaluation should remain in offline validation and test-time reporting because it reduces variance from single-crop predictions. The class-imbalance controls should be retained, specifically the balanced sampler and focal loss combination, since they support minority-class performance. After deployment, class-level drift and quality should be monitored continuously to detect degradation early.

Operationally, we recommend a staged rollout with shadow evaluation before full traffic exposure. During rollout, monitor both confidence calibration and per-class confusion profiles. If drift is detected, trigger a retraining cycle using the same reproducible protocol so that new and old models remain directly comparable.

10 Limitations and Future Work

Current performance is constrained by dataset label noise and genre overlap, which limit the achievable upper bound for single-label classification. Minority classes remain harder because

support is uneven across genres, and audio corruption in source files adds additional preprocessing noise.

Future work should evaluate stronger audio encoders under a controlled compute budget to improve representation quality, including transfer-learning setups with pretrained audio backbones [4]. We also plan to introduce calibrated confidence thresholds for abstention so that low-confidence predictions can be flagged for review. Data-centric improvements, including targeted cleaning, class-aware augmentation, and rebalancing strategies, are likely to produce additional gains.

Another priority is richer labeling objectives. Because many tracks may plausibly span multiple stylistic categories, a future multilabel formulation may better match semantic reality than strict single-label prediction. This would require revised evaluation and business acceptance criteria, but may reduce confusion among related genres.

11 References

References

[1] Michaël Defferrard, Kirell Benzi, Pierre Vandergheynst, and Xavier Bresson. FMA: A Dataset for Music Analysis. In *18th International Society for Music Information Retrieval Conference (ISMIR)*, 2017.

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[3] Keunwoo Choi, George Fazekas, Mark Sandler, and Kyunghyun Cho. Transfer Learning for Music Classification and Regression Tasks. In *18th International Society for Music Information Retrieval Conference (ISMIR)*, 2017.

[4] Shawn Hershey et al. CNN Architectures for Large-Scale Audio Classification. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017.

12 Reproducibility Appendix

12.1 Environment

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

12.2 Run Commands

CNN model comparison (full protocol):

```
PATH="$(pwd)/.venv/bin:$PATH" bash scripts/run_cnn_comparison.sh
```

Single-file inference demo:

```
python predict_genre.py fma_large/002/002003.mp3 \
  --checkpoint outputs/cnn_residual_fullprotocol/best_model.pt --top-k 5
```

12.3 Figure/Table Reproduction Map

Artifact in report	Reproduction source
Pipeline diagram (Fig. 3)	Embedded TikZ in this `.tex` file
Model comparison table (Table 2)	`outputs/*_fullprotocol/summary.json`
Training curves (optional figure)	`outputs/<run>/metric_curves.png` generated by training script
Model-level metric comparison figure	`docs/model_comparison_metrics.png` generated from full-protocol `summary.json` files
Confusion matrix analysis	`outputs/<run>/test_confusion_matrix.npy`

12.4 Deployment Monitoring Requirements

If this model is deployed as a service, monitoring should include latency at P50, P95, and P99; throughput measured in requests per second; data drift across duration, sample-rate, and loudness distributions; and prediction drift tracked through per-class output distributions over time.

12.5 Team Contributions

Table 4: Team contribution summary (fill before submission).

Member	Responsibility	Deliverables
Ang Kar Kin	Data + preprocessing	split policy, cache precompute, quality checks
Nicolas Lasch	Modeling + experiments	architecture variants, tuning, evaluation
Nisal Jinadasa	Reporting + deployment demo	report, presentation, inference artifact

12.6 Submission Checklist Pointers

Before final upload, confirm that the report PDF is present in the repository, code and notebooks are documented, dataset and large weights are provided via external link if needed, and reproducibility commands have been validated end-to-end.